

Final Project Proposal – Morphological Filter

Group 4 (Smruti Pawaskar and Ashay Berde)

I. Project plan

Goal

The goal of this project is to design and implement a 2D Morphological Image Filter on the PYNQ-Z2 System on Chip (SoC), leveraging both the Programmable Logic (PL) and the Processing System (PS) components of the chip. Morphological filtering is a key operation in image processing used for the preprocessing/postprocessing of binary images, including operations like dilation, erosion, opening, and closing. These operations are fundamental in applications such as noise removal, image enhancement, and feature extraction. This project aims to accelerate these operations by utilizing the hardware capabilities of the PYNQ-Z2 SoC, offering a blend of software flexibility and hardware performance.

A. Work Statement - Task Listing:

a. System Specification

- Define Filter Operations: Specify the types of morphological operations (dilation, erosion, opening, closing) to be implemented.
- Image Input/Output Requirements: Determine the format and size of images to be processed, and the method for displaying or outputting the processed images.

b. Hardware Design

- Video Processing Unit: Design and implement VHDL modules for performing morphological operations directly on image data, optimized for high throughput.
- Image Input/Output Interface: Develop an interface in VHDL for loading images into the FPGA and retrieving processed images, possibly including DMA for efficient data transfer.

c. Software Design

- User Interface: Create a Python-based user interface for loading images, selecting morphological operations, and adjusting parameters like structuring element size and shape.
- Integration with Hardware: Develop software routines to send control signals and image data to the hardware modules, and to receive and display the processed images.

d. Testing

- Hardware Component Testing: Utilize VHDL test benches to verify the correct implementation of each morphological operation in isolation.
- Integrated System Simulation: Use simulation tools to test the full system, including both hardware and software components, for a range of images and operations.

- Physical Testing: Test the complete system on a PYNQ-Z2 board with real images, monitoring for both functionality and performance.

e. Documentation

- Document the design process, including both hardware and software components.
- Prepare a final report summarizing the project outcomes and learning experiences.

B. Projected Schedule with Milestones

Week	Milestone	Expected Completion Dates	Actual Completion Dates	Responsible
1	Hardware specifications defined and initial VHDL module design for different morphological operations.	04/12/24	04/12/24	Smruti
2	Completion of VHDL modules for dilation and erosion.	04/17/24	4/18/24	Ashay
3	Basic Python interface for image input/output and hardware control.	04/19/24	04/20/24	Smruti
4	Integration of hardware and software testing with basic morphological operations. Prepare the final presentation. Finalize Report.	04/22/24	04/22/24	Ashay Smruti Ashay

C. Resource Requirements

- Two team members involved in hardware, software, and documentation.
- Hardware: PYNQ-Z2 board, external HDMI display.
- Software: Vivado (2018 or newer) for hardware design and simulation, Python (Jupyter notebook) environment for software.
- Time: 5 weeks of access to the PYNQ-Z2 board and development tools.

D. Previous Work in the Area

Previous implementations of morphological filters on FPGAs have shown significant performance improvements over purely software-based solutions. However, many of these solutions were not flexible in terms of algorithm selection or parameter adjustment. This project aims to differentiate itself by offering a dynamic interface for real-time adjustment of filter parameters and selection between different morphological operations, making it more versatile for various image processing applications. The integration with the PYNQ framework will also simplify the development and usage of the morphological filter, making it accessible to a wider range of users and applications.

II. Work Accomplished

We started our base as lab project 5 which can be seen from the figure 1 below. We then have added switch in this design with reference to select which operations to be performed.

► Hardware Setup

Configuration of the PYNQ board and setup necessary connections for image inputs and outputs.

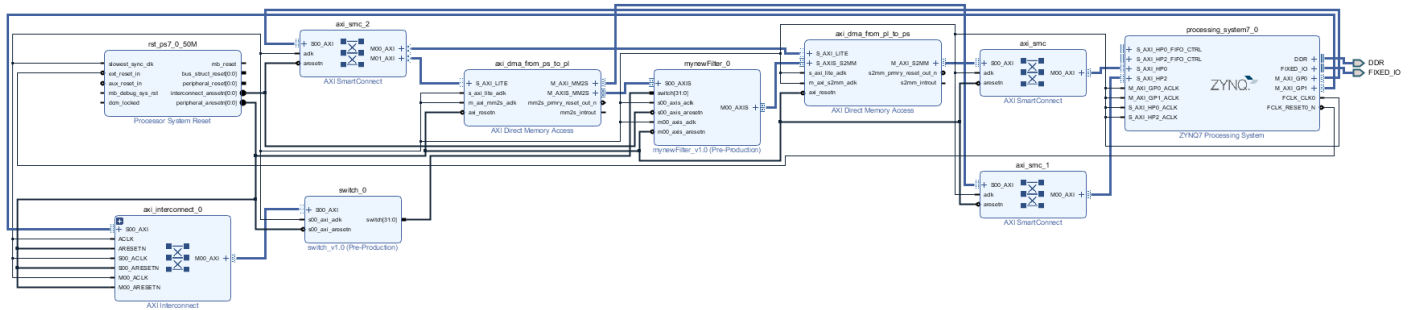


Figure 1: Block Diagram

This block diagram represents system's architecture built on the PYNQ-Z2 platform, showcasing the path of image data through morphological operations processing pipeline. The ZYNQ7 Processing System is the central processing unit of the PYNQ-Z2 board. It interfaces with the DDR memory and manages overall system operations, including running the Jupyter Notebook that allows for interactive development. AXI GPIO block, which typically interfaces with general-purpose input/output pins. This is where the system receives signals to trigger processing or read settings. An AXI Interconnect is a bus interface that allows multiple AXI interfaces to communicate with each other, enabling different blocks of the design to transfer data efficiently. The AXI DMA blocks are responsible for direct memory access, efficiently transferring data between the system memory and the processing elements without burdening the processor. Data seems to flow from DMA to custom IP blocks where morphological operations such as erosion, dilation, opening, and closing are performed. These custom IP blocks process the data and then transfer it back via DMA.

Basics of Morphological Operations

In the context of image processing, these are mathematical operations applied to the structure or shape of features in an image. They are particularly useful in the field of computer vision and image analysis for tasks such as noise reduction, shape refinement, and feature extraction.

1. Erosion: It is akin to "sandblasting" an image. It systematically strips away the outermost layer of pixels from shapes in the image. It reduces the size of objects, eliminates small anomalies, and disconnects objects that are barely touching. Useful for removing small bits of noise and separating objects that are close to each other.

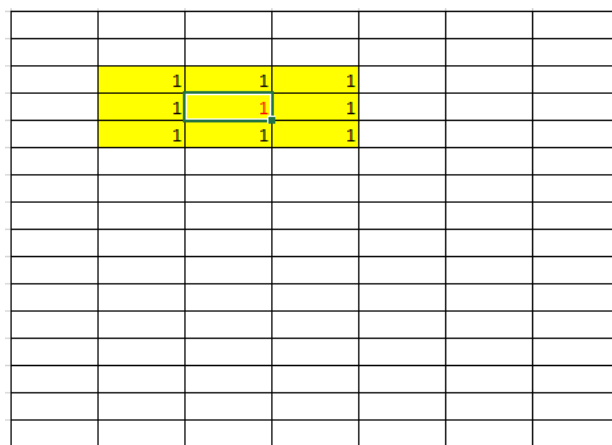
2. Dilation: It does the opposite of erosion. Imagine each pixel expanding outwards or "blooming". It increases the size of objects, fills in small holes, and connects objects that are near each other. Often used after erosion to restore the original size of objects while keeping the noise out.

3. Opening: It is a two-step process that starts with erosion followed by dilation. Think of it as an "open space" operation that clears away small objects. It removes small objects and noise from an image while maintaining the shape and size of larger objects. It's great for clearing away small imperfections or specks within or around objects.

4. Closing: It is the reverse of opening: it begins with dilation followed by erosion. It's like closing the gaps in an image. It fills in small holes and gaps in the objects and connects nearby objects. Useful for closing small holes inside objects or joining objects that are close together.

Morphological operations can be tailored with different structuring elements (the shape used to probe the image) to achieve specific effects on the image.

We started by implementing dilation on the input and then erosion. After the successful implementation of both these operations. We combined them one after the other for the opening and closing operations. We have used a 3 by 3 window with all 1's as a structuring element.



		1		1		1								
		1		1		1								
		1		1		1								

Figure 2: Structuring Element

We need to find the maximum value of all the input pixels' neighborhoods for Dilation. The minimum value of all the input pixels neighborhood for Erosion.

For Opening and Closing, we need to take the output of the previous operation and then feed it as input to the next. Here is a snippet of how we implemented it in VHDL.

```

begin
  -- Instantiation of the Erosion_Dilation
  Erosion_Dilation : FIR_3x3
  port map (
    clk => s00_axis_aclk,
    rst => s00_axis_aresetn,
    switch => switch,
    run => run,
    x_input => datain,
    y => out_signed_erosiondilation
  );
  Open_Close : FIR_3x3_1
  port map (
    clk => s00_axis_aclk,
    rst => s00_axis_aresetn,
    switch => switch,
    run => run,
    x_input => out_signed_erosiondilation(8 downto 0),
    y => out_signed_horizontall
  );
end;

```

Figure 3 : Implementation logic for Opening and Closing Operations

```

-- Perform morphological operations
IF switch =0 THEN -- Erosion
  min_val := window(1, 1); -- Initialize minimum value
  FOR i IN 1 TO 3 LOOP
    FOR j IN 1 TO 3 LOOP
      IF window(i, j) < min_val THEN
        min_val := window(i, j); -- Find minimum
      END IF;
    END LOOP;
  END LOOP;
  y <= min_val; -- Assign resized min value to output
ELSIF switch =1 THEN -- Dilation
  max_val := window(1, 1); -- Initialize maximum value
  FOR i IN 1 TO 3 LOOP
    FOR j IN 1 TO 3 LOOP
      IF window(i, j) > max_val THEN
        max_val := window(i, j); -- Find maximum
      END IF;
    END LOOP;
  END LOOP;
  y <= max_val; -- Assign resized max value to output

```

Figure 4: Implementation logic for Dilation and Erosion

Code Logic

➤ Key Components:

mynewFilter_v1_0_S00_AXIS and mynewFilter_v1_0_M00_AXIS are components designed to handle AXI data transfers efficiently.

➤ Instance Creation:

2 instances **FIR_3x3** and **FIR_3x3_1** are created for Key processing units for applying morphological filters, and handling different data resolutions.

➤ **Instance Mapping:**

The output of the first instance is mapped to input of the second instances based upon the switching logic if the switch is set to 0 or 1 (Erosion or Dilation) then the output of the first instance is directly sent to the output buffer.

➤ **Detailed Morphological Operations**

- **Switch Settings and Operations**

1. Switch = 0: Erosion

Process Flow:

Erosion is implemented by the FIR_3x3 component which is our 1st instance. It involves scanning the 3x3 pixel window and selecting the minimum pixel value within that area. This effectively reduces the size of objects and removes smaller details.

The minimum finding algorithm iterates over each pixel in the 3x3 matrix (window), compares each pixel value to the current minimum (min_val), and updates min_val if a smaller value is found. A comparison-based logic is used to find the minimum value.

The final minimum value is output to y, and is resized from 9 bits to 18 bits with zero padding as the output port width is 18-bits representing the result of the erosion process.

2. Switch = 1: Dilation

Process Flow:

Dilation is executed by the FIR_3x3 component. It scans the 3x3 pixel window, identifying the maximum value within this neighborhood to increase the prominence of features.

The maximum value is determined by iterating through each pixel in the window, comparing each value against the current maximum (max_val), and updating max_val whenever a greater value is encountered.

The resultant maximum value is then output to y, denoting the dilation result.

3. Switch = 2: Opening

Process Flow:

Opening is handled as a sequential operation of erosion followed by dilation. Initially, the FIR_3x3 performs erosion, reducing noise and removing small objects. The result of this erosion serves as the input to the FIR_3x3_1, which then applies dilation. This sequence effectively removes small objects and noise while preserving the overall shape and size of larger objects.

4. Switch = 3: Closing

Process Flow:

Closing involves an initial dilation process to close small gaps and connect close objects, followed by an erosion to smooth the boundaries of enlarged objects. Dilation is first executed by the FIR_3x3 Filter instance, and the result is subsequently eroded by FIR_3x3_1 Filter instance.

➤ Signal Flow and Data Processing

Data Input/Output:

Input data (s00_axis_tdata) is initially resized to match the processing requirements of the FIR filters. It is then processed based on the active switch setting.

The output from the FIR filters is resized and formatted to match the AXI master interface's expectations, ensuring correct data alignment and transmission.

We have created 2 instances of filter based upon switching operations the first instance performs Erosion and Dilation, the second instances is used for performing Opening and closing.

➤ Software Development

Use of Python programming in Jupyter Notebook and OpenCV for implementing morphological operations.
Development of user interface for interactive operation control.

The switch is used to choose between the four operations. It has AXI-Lite interfacing so it has four register each of 32 width. We are controlling the switch using the jupyter notebook and we have used lab project 3 as a reference for it.

```
In [6]: overlay?

In [7]: switch= overlay.switch_0
        switch?

In [8]: switch.write(0x08,3)
        # switching between different morphological operations
        # insert 0 gives Erosion, 1 gives Dilation, 2 gives Opening, 3 gives Closing
```

Figure 5: Switch used in Jupyter Notebook

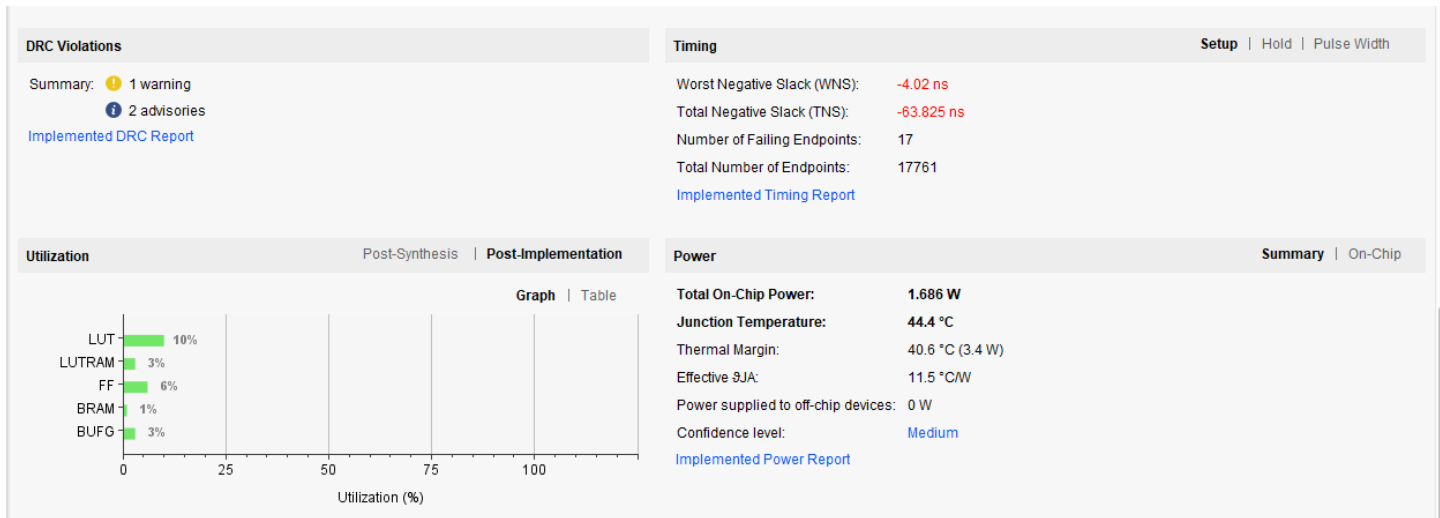


Figure 6: Project summary

- Challenges faced during the project :
 1. Initially did not get proper output for opening/closing(bits mismatch)
 2. Tried to implement the delay in hardware, instead did it on Jupyter notebook
- Debugging rules that helped us resolve the issues:
 - Understand the System
 - Keep an Audit Trail

III. Results

Presentation of before and after images to show the effects of the morphological operations.

- Erosion



Figure 7: We can see visible difference in the Mustache of tiger

The middle image is labeled as "Hardware Image" which suggests that it is the result of an erosion operation performed using hardware-accelerated PYNQ-Z2.

Upon detailed examination, it exhibits the typical effects of erosion:

Contrast with the Original: Compared to the original image on the left, the middle image shows a reduced level of detail. The fur texture is less pronounced, and the edges of the tiger's features, such as its stripes, ears, and facial contours, appear smoother.

Noise Reduction: If there were any noise elements in the original image, they seem to have been diminished in the eroded version, contributing to a cleaner appearance though at the cost of fine detail.

Boundary Effects: The outlines of the tiger's features, especially around the whiskers and the fur, have receded slightly. This is a classic sign of erosion, where the boundary pixels that do not fit the structuring element's shape are transformed to match the background.

Object Separation and Smoothing: While the image does not show closely touching objects that would be separated by erosion, the general smoothing effect on the tiger's fur and the background elements illustrates the smoothing capability of erosion.

➤ Dilation



Figure 8: Fills in the gap from the original image

The middle and the right image show the effects of dilation, as applied through both hardware and software techniques. Dilation, which is essentially the inverse operation to erosion, adds pixels to the boundaries of objects in an image, thus increasing their size and accentuating features.

Upon detailed examination, it exhibits the typical effects of Dilation:

Hardware Dilation (Middle Image): This image demonstrates a moderate level of detail enhancement. The fur texture and facial features of the tiger, such as the stripes and whiskers, have become more prominent compared to the original image. The edges appear smoother, which is a sign that the dilation has been performed uniformly.

Software Dilation (Right Image): The software-processed image exhibits a sharper and more distinct increase in the size of the features. There's a noticeable contrast enhancement which indicates a more aggressive dilation, or the use of a larger structuring element. This has resulted in thicker stripes and a fuller appearance of the tiger's fur.

In both dilated images, small gaps or breaks in the object's features would be filled in, making them appear more continuous. This effect is useful for reconnecting areas of objects that may have been disconnected due to prior erosion or other processing.. The comparison between the hardware and software dilated images may also reflect differences in the algorithms or structuring elements used in the dilation process.

➤ Opening



Figure 9: Erosion followed by Dilation

Hardware Opening (Middle Image):

Removal of Small Details: The middle image shows a reduction in the finer details of the tiger's fur compared to the original image. This suggests that the initial erosion step of the opening process has effectively removed these small features.

Retention of Main Features: Despite the softening of fine details, the primary features of the tiger, such as the major stripes and facial outlines, remain intact and are clearly visible. This indicates the subsequent dilation step has restored the shape of the larger structures after the initial erosion.

Compare the hardware and Software images

```
In [870]: # Create a figure to hold the subplots
plt.figure(figsize=(30, 10)) # Adjust the size as needed to accommodate three subplots

# First subplot for the original image
plt.subplot(1, 3, 1) # 1 row, 3 columns, first subplot
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off') # This turns off the axis labels and ticks

# Second subplot for the first image (cropped)
plt.subplot(1, 3, 2) # 1 row, 3 columns, second subplot
plt.imshow(cropped_image_array, cmap='gray')
plt.title('Hardware Image')
plt.axis('off') # This turns off the axis labels and ticks

# Third subplot for the second image (cropped and eroded)
plt.subplot(1, 3, 3) # 1 row, 3 columns, third subplot
plt.imshow(cropped_closed_img, cmap='gray')
plt.title('Software Image')
plt.axis('off') # This turns off the axis labels and ticks

# Display the plot
plt.show()
```



Figure 10: Closing Operation on input image of Cat and comparing it with Ground truth

IV. Recommendations & Future work.

Milestone tasks if someone was trying to replicate this project

Performing Dilation and Erosion Correctly
Adding switch conditions
Performing Opening and Closing Correctly
Comparing the Hardware and Software Images

Future Scope

- Can use higher structuring elements (5 by 5, 7 by 7) or different structuring elements (plus sign).
- Can display the output on the monitor.

Appendices

Results of the earlier approach

The approach we used earlier started from lab5 where we implemented Horizontal and Vertical Filters separately so we first did the horizontal erosion, dilation, opening and closing and then input this the to the vertical filter getting the results below. The result seen show very minimal morphological operations result the kernel implemented was 3x3. So we went ahead and tried to do 3x3 inseparable mask.

Original Image:



Eroded Image:



Dilated Image:



Opening Image:



Close Image:

